

Minimal assistant-ui Chat UI Setup

Smallest practical App.tsx before adding the Corporate Policy Assistant layout

Goal: Verify that assistant-ui works in the React TypeScript/Vite project before adding Azure integration, RAG, authentication, branding, or the full Corporate Policy Assistant interface.

1. Recommended minimal approach

Keep App.tsx intentionally small. It only needs:

- an assistant-ui runtime
- the runtime provider
- the assistant-ui Thread component
- a temporary fake response so the chat can be tested without Azure or a backend

This lets assistant-ui manage the chat experience while the model adapter acts as a temporary stand-in for the future REST API/backend.

2. Replace all of src/App.tsx

```
import {
  AssistantRuntimeProvider,
  useLocalRuntime,
  type ChatModelAdapter,
} from "@assistant-ui/react";

import { Thread } from "../components/assistant-ui/thread";

const modelAdapter: ChatModelAdapter = {
  async run() {
    return {
      content: [
        {
          type: "text",
          text: "Hello! The assistant-ui chat interface is working.",
        },
      ],
    };
  },
};

function App() {
  const runtime = useLocalRuntime(modelAdapter);

  return (
    <AssistantRuntimeProvider runtime={runtime}>
      <Thread />
    </AssistantRuntimeProvider>
  );
}

export default App;
```

Important: Do not delete src/components/assistant-ui/thread.tsx. That component contains the assistant-ui thread implementation and is what keeps App.tsx small.

3. What each part does

Imports assistant-ui

```
import {
  AssistantRuntimeProvider,
  useLocalRuntime,
  type ChatModelAdapter,
} from "@assistant-ui/react";
```

These imports provide the runtime, the provider that exposes it to the UI, and the adapter type used for model/backend communication.

Imports the chat thread

```
import { Thread } from "../components/assistant-ui/thread";
```

Thread is the visible chat interface: messages, composer, send behavior, scrolling, and related assistant-ui interaction elements.

Temporary fake AI response

```
const modelAdapter: ChatModelAdapter = {
  async run() {
    return {
      content: [
        {
          type: "text",
          text: "Hello! The assistant-ui chat interface is working.",
        },
      ],
    };
  },
};
```

For now, every message receives the same response. This is intentional: the objective is to prove that the assistant-ui front end is wired correctly before introducing backend complexity.

Connects the runtime to the Thread

```
function App() {
  const runtime = useLocalRuntime(modelAdapter);

  return (
    <AssistantRuntimeProvider runtime={runtime}>
      <Thread />
    </AssistantRuntimeProvider>
  );
}
```

The local runtime manages the chat state and calls the model adapter whenever the user sends a message.

4. Current test flow

```
User types a question
|
v
assistant-ui Thread
|
v
Local Runtime
|
v
modelAdapter
|
v
Static test response
```

There is no Azure connection at this stage. That is the desired state for this checkpoint.

5. Run the project

From the React project directory, run:

```
npm run dev
```

Then open the Vite development URL shown in the terminal, commonly:

```
http://localhost:5173
```

Type a test question such as:

```
What is the corporate remote work policy?
```

The assistant should respond:

```
Hello! The assistant-ui chat interface is working.
```

The response will be identical regardless of the question. That confirms the UI/runtime path is working.

6. Do not add these yet

- Azure Functions
- Azure AI Search
- Microsoft Foundry
- LLM/RAG integration
- REST API calls
- Authentication
- Corporate Policy branding
- Sidebar or conversation history
- File uploads

- Custom message components

7. Checkpoint architecture

```
React / Vite
|
v
assistant-ui
|
v
Local Runtime
|
v
Fake Model Adapter
|
v
Static Response
```

8. Next step after this works

Once the minimal chat works reliably, make one controlled change: replace the fake model adapter response with a call to the Corporate Policy Assistant REST API/backend.

```
assistant-ui
|
v
Local Runtime / Model Adapter
|
v
Corporate Policy REST API
|
v
Azure backend / AI Search / Foundry
|
v
Assistant response
```

This keeps troubleshooting isolated: first prove the front end, then add the backend, then add the full Corporate Policy Assistant layout and enterprise features.